

Lembar Materi & Praktikum: File Upload (Local & Cloud), Email Service, dan Dokumentasi API Swagger

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Praktikum ini melengkapi materi teori pada [16. swagger-api.md](#). Anda akan dipandu untuk mengimplementasikan tiga fitur krusial yang melengkapi ekosistem API modern pada proyek **Sistem Tiket Konser** (`go-tiket-konser`) menggunakan framework **Gin** dan **GORM**. Kita akan membangun fitur unggah berkas (poster konser) dengan validasi ketat, merancang abstraksi penyimpanan (*local & cloud*) berbasis **Strategy Pattern**, mengirimkan email konfirmasi tiket secara *asynchronous* (non-blocking) menggunakan **Goroutines**, serta mendokumentasikan seluruh API menggunakan anotasi **Swagger (Swaggo)**.

Sesi 1: File Upload di Gin & Abstraksi Penyimpanan (Local vs Cloud) (45 Menit)

1. File Upload Menggunakan Gin

Dalam protokol HTTP, pengunggahan berkas dilakukan melalui format **Multipart/Form-Data**, bukan JSON. Gin memfasilitasi penangkapan file ini dengan method `c.FormFile("key_name")` yang mengembalikan objek `*multipart.FileHeader`.

Poin-Poin Validasi Krusial (Keamanan):

- **Batas Ukuran (Size Limit):** Mencegah pengguna mengunggah berkas raksasa yang dapat menghabiskan ruang penyimpanan server (serangan *Denial of Service*).
- **Validasi Ekstensi / MIME Type:** Memastikan hanya jenis file gambar (seperti JPG, JPEG, atau PNG) yang diperbolehkan. Ini mencegah penyerang mengunggah berkas executable berbahaya (seperti `.exe` atau `.sh`).
- **Sanitasi Nama File (Unique Naming):** Mengubah nama asli berkas menjadi nama unik berbasis timestamp atau UUID untuk mencegah file tertimpa (*overwritten*) jika ada dua user mengunggah berkas bernama sama.

2. Abstraksi Penyimpanan (Strategy Pattern)

- **Penyimpanan Lokal (Local Storage):** Berkas disimpan langsung di harddisk server tempat aplikasi backend berjalan.
 - *Kelebihan:* Sangat mudah diimplementasikan, gratis, kueri cepat.
 - *Kekurangan:* Tidak cocok untuk arsitektur server terdistribusi (multi-instance). Jika server backend dideploy ulang di platform cloud stateless (seperti Heroku atau AWS Fargate), berkas lokal akan terhapus.
- **Penyimpanan Awan (Cloud Storage):** Berkas dikirim langsung ke penyedia Object Storage eksternal seperti AWS S3, Google Cloud Storage, atau Cloudinary.
 - *Kelebihan:* Stateless, andal, terdistribusi global via CDN, file aman walau server backend hancur atau dideploy ulang.
 - *Kekurangan:* Latensi pengiriman file bertambah, membutuhkan konfigurasi API key, biaya tambahan.

Strategy Pattern dengan Interface: Agar backend kita dapat berganti-ganti dari lokal ke cloud (misal: lokal saat testing, AWS S3 saat produksi) tanpa mengubah satu baris pun kode di level *Handler*, kita wajib membungkus logika penyimpanan dalam sebuah **Interface Go**:

```
package service

import "mime/multipart"

// StorageProvider mendefinisikan kontrak pengunggahan berkas ke sistem penyimpanan
type StorageProvider interface {
    UploadFile(file *multipart.FileHeader) (string, error)
}
```

Sesi 2: Layanan Notifikasi Email (SMTP Service) secara Asynchronous (45 Menit)

1. Protokol SMTP dan Email Service di Go

Go menyediakan paket bawaan `"net/smtp"` untuk berkomunikasi dengan server surat. Kita dapat menggunakannya bersama penyedia SMTP simulasi untuk pengujian seperti **Mailtrap.io** atau SMTP asli (Gmail App Password, Mailgun, SendGrid).

2. Pengiriman Email Non-Blocking (Asynchronous)

Menghubungi server SMTP eksternal membutuhkan waktu sekitar 1 hingga 3 detik karena proses jabat tangan (*TCP handshake*) dan autentikasi SSL/TLS.

WARNING

Jika kita mengirim email secara sinkron (*synchronous*) di dalam siklus request-response HTTP, pengguna harus menunggu 3 detik ekstra di layar browser sebelum mendapatkan respon sukses pemesanan tiket. Ini adalah pengalaman pengguna (UX) yang sangat buruk!

Solusi: Jalankan pengiriman email di latar belakang (*background job*) menggunakan **Goroutine** (`go emailService.SendMail(...)`). Dengan ini, respon HTTP `201 Created` langsung dikirimkan ke pengguna, sementara pengiriman email diproses secara mandiri oleh mesin penjadwalan Go tanpa menahan request klien.

Sesi 3: API Documentation dengan Swagger (Swaggo) (45 Menit)

1. Mengapa Perlu Dokumentasi API Interaktif?

Dokumentasi API manual (seperti menulis file PDF atau Word) sangat cepat usang begitu ada perubahan kode. **Swagger (OpenAPI)** menengahi masalah ini dengan menghasilkan dokumentasi interaktif berbasis web langsung dari komentar anotasi (*Go Doc Comments*) di atas kode backend Anda.

2. Memahami Anotasi Swagger Utama

Swaggo menerjemahkan komentar bertanda `@` menjadi spesifikasi OpenAPI:

- `@Summary` : Deskripsi singkat apa yang dilakukan endpoint.
 - `@Description` : Detail perilaku atau alur endpoint.
 - `@Tags` : Mengelompokkan endpoint (misal: "Concerts", "Auth", "Bookings").
 - `@Accept & @Produce` : Format data masukan dan luaran (biasanya `json` atau `mpfd` untuk multipart form-data).
 - `@Param` : Parameter masukan (query, path, body, atau form-data). Format: `nama type dataType required "keterangan"` .
 - `@Success & @Failure` : Contoh respons sukses atau gagal, lengkap dengan struct DTO target.
 - `@Router` : URL path dan method HTTP. Format: `/api/v1/concerts [get]` .
-

Sesi 4: Praktikum Mandiri – Langkah Integrasi Proyek Konser (45 Menit)

Mari kita pelajari cara menyusun implementasi ini pada proyek **Sistem Tiket Konser** Anda.

1. Desain Model Tambahan Poster, Thumbnail, & Tata Tertib Konser:

[models/concert.go](#)

Kita tambahkan field `PosterURL`, `ThumbnailURL`, dan `RulesPDFURL` untuk menyimpan tautan gambar poster, gambar thumbnail, serta berkas PDF tata tertib konser ke dalam database:

```
type Concert struct {
    ID          int          `gorm:"primaryKey;autoIncrement" json:"id"`
    Title       string       `gorm:"not null" json:"title"`
    Description  string       `gorm:"type:text" json:"description"`
    Date        time.Time   `gorm:"not null" json:"date"`
    Venue       string       `gorm:"not null" json:"venue"`
    Status      string       `gorm:"default:active" json:"status"`
    PosterURL   string       `gorm:"type:varchar(255)" json:"poster_url" //
    // Field untuk poster
    ThumbnailURL string      `gorm:"type:varchar(255)" json:"thumbnail_url" //
    // Field baru untuk thumbnail
    RulesPDFURL string      `gorm:"type:varchar(255)" json:"rules_pdf_url" //
    // Field baru untuk PDF tata tertib
    CreatedAt    time.Time   `gorm:"autoCreateTime" json:"created_at"`
    UpdatedAt    time.Time   `gorm:"autoUpdateTime" json:"updated_at"`
}
```

2. Implementasi Interface Penyimpanan: [service/storage.go](#)

Buat berkas baru bernama `storage.go` di bawah folder `service/` untuk mendefinisikan interface dan penyedia penyimpanan lokal:

```
package service

import (
    "errors"
    "fmt"
    "mime/multipart"
    "os"
    "path/filepath"
    "strings"
    "time"

    "github.com/gin-gonic/gin"
)
```

```

type StorageProvider interface {
    UploadFile(file *multipart.FileHeader, c *gin.Context) (string, error)
}

type localStorageProvider struct {
    UploadDir string
    BaseURL   string
}

func NewLocalStorageProvider(uploadDir string, baseUrl string) StorageProvider {
    // Pastikan folder penyimpanan lokal terbentuk
    _ = os.MkdirAll(uploadDir, os.ModePerm)
    return &localStorageProvider{
        UploadDir: uploadDir,
        BaseURL:   baseUrl,
    }
}

func (l *localStorageProvider) UploadFile(file *multipart.FileHeader, c *gin.Context) (string, error) {
    // 1. Validasi Batas Ukuran File (Maksimal 2 MB)
    var maxFileSize int64 = 2 * 1024 * 1024 // 2MB
    if file.Size > maxFileSize {
        return "", errors.New("ukuran file melebihi batas maksimum 2MB")
    }

    // 2. Validasi Ekstensi Gambar & PDF Tata Tertib
    ext := strings.ToLower(filepath.Ext(file.Filename))
    if ext != ".jpg" && ext != ".jpeg" && ext != ".png" && ext != ".pdf" {
        return "", errors.New("format file tidak didukung: hanya boleh JPG, JPEG, PNG, atau PDF")
    }

    // 3. Buat Nama Unik
    fileName := fmt.Sprintf("%d%s", time.Now().UnixNano(), ext)
    filePath := filepath.Join(l.UploadDir, fileName)

    // 4. Simpan ke sistem berkas server lokal
    if err := c.SaveUploadedFile(file, filePath); err != nil {
        return "", fmt.Errorf("gagal menulis file ke disk: %v", err)
    }

    // 5. Kembalikan URL publik
    return fmt.Sprintf("%s/uploads/%s", l.BaseURL, fileName), nil
}

```

3. Implementasi Handler Upload Thumbnail Konser: [handler/concert.go](#)

Buat handler untuk menangkap file dari request multipart form (dengan key name `thumbnail`), memanggil `StorageProvider` untuk menyimpan gambar, dan memperbarui field `ThumbnailURL`

dari konser terkait di database:

```
package handler

import (
    "net/http"
    "strconv"

    "go-tiket-konser/dto"
    "go-tiket-konser/service"

    "github.com/gin-gonic/gin"
)

type ConcertHandler struct {
    service          service.ConcertService
    storageProvider  service.StorageProvider
}

func NewConcertHandler(s service.ConcertService, sp service.StorageProvider)
*ConcertHandler {
    return &ConcertHandler{service: s, storageProvider: sp}
}

// UploadThumbnail handles POST /api/v1/concerts/:id/thumbnail
func (h *ConcertHandler) UploadThumbnail(c *gin.Context) {
    idStr := c.Param("id")
    id, err := strconv.Atoi(idStr)
    if err != nil {
        c.JSON(http.StatusBadRequest, dto.WebResponse{Success: false, Message:
"ID Konser tidak valid"})
        return
    }

    // 1. Ambil file thumbnail dari form request
    file, err := c.FormFile("thumbnail")
    if err != nil {
        c.JSON(http.StatusBadRequest, dto.WebResponse{Success: false, Message:
"Berkas thumbnail wajib dikirim"})
        return
    }

    // 2. Simpan file menggunakan StorageProvider (Local/Cloud)
    fileURL, err := h.storageProvider.UploadFile(file, c)
    if err != nil {
        c.JSON(http.StatusBadRequest, dto.WebResponse{Success: false, Message:
err.Error()})
        return
    }

    // 3. Update field ThumbnailURL di database
    concert, err := h.service.GetConcertByID(id)
```

```

        if err != nil {
            c.JSON(http.StatusNotFound, dto.WebResponse{Success: false, Message:
"Konser tidak ditemukan"})
            return
        }

        concert.ThumbnailURL = fileURL
        err = h.service.UpdateConcert(&concert)
        if err != nil {
            c.JSON(http.StatusInternalServerError, dto.WebResponse{Success: false,
Message: "Gagal menyimpan data thumbnail"})
            return
        }

        c.JSON(http.StatusOK, dto.WebResponse{
            Success: true,
            Message: "Thumbnail konser berhasil diunggah dan disimpan",
            Data:    fileURL,
        })
    }

    // UploadRulesPDF handles POST /api/v1/concerts/:id/rules
    func (h *ConcertHandler) UploadRulesPDF(c *gin.Context) {
        idStr := c.Param("id")
        id, err := strconv.Atoi(idStr)
        if err != nil {
            c.JSON(http.StatusBadRequest, dto.WebResponse{Success: false, Message:
"ID Konser tidak valid"})
            return
        }

        // 1. Ambil berkas PDF dari request form dengan key name "rules_pdf"
        file, err := c.FormFile("rules_pdf")
        if err != nil {
            c.JSON(http.StatusBadRequest, dto.WebResponse{Success: false, Message:
"Berkas PDF tata tertib wajib dikirim"})
            return
        }

        // 2. Validasi Ekstensi Berkas (Khusus PDF tata tertib)
        ext := strings.ToLower(filepath.Ext(file.Filename))
        if ext != ".pdf" {
            c.JSON(http.StatusBadRequest, dto.WebResponse{Success: false, Message:
"Format berkas tidak didukung: hanya diperbolehkan PDF (.pdf)"})
            return
        }

        // 3. Simpan berkas PDF menggunakan StorageProvider (Local/Cloud)
        fileURL, err := h.storageProvider.UploadFile(file, c)
        if err != nil {
            c.JSON(http.StatusBadRequest, dto.WebResponse{Success: false, Message:
err.Error()})
            return
        }
    }

```

```

    }

    // 4. Perbarui field RulesPDFURL di database konser
    concert, err := h.service.GetConcertByID(id)
    if err != nil {
        c.JSON(http.StatusNotFound, dto.WebResponse{Success: false, Message:
"Konser tidak ditemukan"})
        return
    }

    concert.RulesPDFURL = fileURL
    err = h.service.UpdateConcert(&concert)
    if err != nil {
        c.JSON(http.StatusInternalServerError, dto.WebResponse{Success: false,
Message: "Gagal menyimpan berkas PDF tata tertib"})
        return
    }

    c.JSON(http.StatusOK, dto.WebResponse{
        Success: true,
        Message: "PDF Tata tertib konser berhasil diunggah dan disimpan",
        Data:    fileURL,
    })
}

```

5. Implementasi Layanan Email SMTP: [service/email.go](https://github.com/indrawati123/service_email.go)

Buat berkas baru bernama `email.go` di bawah folder `service/` untuk mengintegrasikan pengiriman email tiket konser:

```

package service

import (
    "fmt"
    "net/smtp"
)

type EmailService interface {
    SendTicketConfirmation(to string, customerName string, bookingCode string,
totalAmount float64) error
}

type emailService struct {
    SMTPHost    string
    SMTPPort    string
    SenderEmail string
    AuthPassword string
}

func NewEmailService(host string, port string, sender string, password string)

```



```

EmailService {
    return &emailService{
        SMTPHost:    host,
        SMTPPort:    port,
        SenderEmail:  sender,
        AuthPassword: password,
    }
}

func (s *emailService) SendTicketConfirmation(to string, customerName string,
bookingCode string, totalAmount float64) error {
    // 1. Mengatur Subject dan Body HTML
    subject := "Konfirmasi Pemesanan Tiket Konser: " + bookingCode
    body := fmt.Sprintf(`
        <html>
        <body>
            <h2>Halo, %s!</h2>
            <p>Terima kasih telah memesan tiket di platform kami.</p>
            <p>Berikut adalah detail pesanan Anda:</p>
            <ul>
                <li><b>Kode Booking:</b> %s</li>
                <li><b>Total Pembayaran:</b> Rp %.2f</li>
                <li><b>Status:</b> Lunas</li>
            </ul>
            <p>Silakan bawa kode booking ini ke lapangan untuk ditukarkan
dengan gelang tiket fisik.</p>
            <br/>
            <p>Salam hangat,<br/>Tim Konser JuaraCoding</p>
        </body>
        </html>
    `, customerName, bookingCode, totalAmount)

    // 2. Format RFC 822 Message
    message := []byte("Subject: " + subject + "\r\n" +
        "To: " + to + "\r\n" +
        "MIME-Version: 1.0\r\n" +
        "Content-Type: text/html; charset=UTF-8\r\n\r\n" +
        body)

    // 3. Autentikasi SMTP
    auth := smtp.PlainAuth("", s.SenderEmail, s.AuthPassword, s.SMTPHost)

    // 4. Kirim Email
    addr := fmt.Sprintf("%s:%s", s.SMTPHost, s.SMTPPort)
    err := smtp.SendMail(addr, auth, s.SenderEmail, []string{to}, message)
    if err != nil {
        return fmt.Errorf("smtp error: %v", err)
    }

    return nil
}

```

6. Pemicu Asynchronous Email pada Modul Booking: [service/booking.go](https://github.com/indrawati/service/tree/master/service/booking.go)

Buka file service booking Anda, dan masukkan email service ke dalam dependencies. Trigger email konfirmasi secara asynchronous saat transaksi selesai:

```
type bookingService struct {
    db          *gorm.DB
    bookingRepo repository.BookingRepository
    customerRepo repository.CustomerRepository
    emailService EmailService // Suntikan Dependency Email
}

// ... di dalam method CreateBooking setelah transaksi berhasil commit:
func (s *bookingService) CreateBooking(req *dto.BookingRequest) (models.Booking,
error) {
    // ... (proses booking & db.Transaction commit berhasil) ...

    // Ambil data pembeli untuk mendapatkan alamat email
    customer, _ := s.customerRepo.FindByID(req.CustomerID)

    // Kirim email konfirmasi secara Asynchronous (Non-Blocking) menggunakan kata
    kunci 'go'
    go func() {
        errEmail := s.emailService.SendTicketConfirmation(
            customer.Email,
            customer.Name,
            finalBooking.BookingCode,
            finalBooking.TotalAmount,
        )
        if errEmail != nil {
            fmt.Printf("Gagal mengirim email background: %v\n", errEmail)
        }
    }()

    return finalBooking, nil
}
```

7. Setup Swaggo Dokumentasi API

Ikuti langkah-langkah di terminal untuk menghasilkan dokumentasi otomatis:

1. Unduh & Pasang Swag CLI:

```
go install github.com/swaggo/swag/cmd/swag@latest
```

2. Pasang Library Integration di Proyek:

```
go get -u github.com/swaggo/gin-swagger
go get -u github.com/swaggo/files
```

3. **Tulis Anotasi di Handler:** Buka file handler Anda (contoh `handler/concert.go`) dan tuliskan keterangan Swagger:

```
// GetConcerts godoc
// @Summary      Mendapatkan Daftar Konser Terpaginasi
// @Description   Mengambil seluruh konser aktif dengan filter pencarian,
//               pengurutan, dan pembatasan halaman
// @Tags         Concerts
// @Accept       json
// @Produce      json
// @Param        search query string false "Cari konser berdasarkan judul
//               atau tempat"
// @Param        page  query int false "Nomor halaman (default: 1)"
// @Param        limit query int false "Jumlah data per halaman
//               (default: 10)"
// @Param        sort  query string false "Opsi urut: date_asc, date_desc,
//               title_asc, title_desc"
// @Success      200 {object} dto.WebResponse{data=[]dto.ConcertResponse}
//               "Daftar konser sukses diambil"
// @Failure      500 {object} dto.WebResponse "Kesalahan Server Internal"
// @Router       /concerts [get]
func (h *ConcertHandler) GetConcerts(c *gin.Context) { ... }
```

4. **Tulis Anotasi Multipart Form (Upload Poster):**

```
// UploadPoster godoc
// @Summary      Mengunggah Gambar Poster Konser
// @Description   Mengunggah poster fisik konser ke server (Maks 2MB,
//               JPG/JPEG/PNG)
// @Tags         Concerts
// @Accept       mpfd
// @Produce      json
// @Param        file_poster formData file true "Berkas gambar poster"
// @Success      200 {object} dto.WebResponse{data=string} "URL Poster
//               sukses dikembalikan"
// @Failure      400 {object} dto.WebResponse "Input file tidak valid"
// @Failure      500 {object} dto.WebResponse "Kesalahan Server"
// @Router       /concerts/upload-poster [post]
func (h *ConcertHandler) UploadPoster(c *gin.Context) { ... }
```

5. **Tulis Anotasi Multipart Form (Upload Thumbnail):**

```
// UploadThumbnail godoc
// @Summary      Mengunggah Gambar Thumbnail Konser
// @Description   Mengunggah thumbnail fisik konser berdasarkan ID konser (Maks
//               2MB, JPG/JPEG/PNG)
// @Tags         Concerts
// @Accept       mpfd
// @Produce      json
// @Param        id path int true "ID Konser"
// @Param        thumbnail formData file true "Berkas gambar thumbnail"
```

```
// @Success      200      {object}  dto.WebResponse{data=string} "URL Thumbnail
sukses disimpan"
// @Failure      400      {object}  dto.WebResponse "Input tidak valid"
// @Failure      404      {object}  dto.WebResponse "Konser tidak ditemukan"
// @Failure      500      {object}  dto.WebResponse "Kesalahan Server"
// @Router       /concerts/{id}/thumbnail [post]
func (h *ConcertHandler) UploadThumbnail(c *gin.Context) { ... }
```

6. Tulis Anotasi Multipart Form (Upload PDF Tata Tertib):

```
// UploadRulesPDF godoc
// @Summary      Mengunggah Berkas PDF Tata Tertib Konser
// @Description  Mengunggah tata tertib resmi konser dalam format PDF berdasarkan
ID konser (Maks 2MB, PDF saja)
// @Tags        Concerts
// @Accept      mpfd
// @Produce     json
// @Param       id          path      int    true  "ID Konser"
// @Param       rules_pdf   formData  file   true  "Berkas PDF Tata Tertib"
// @Success     200         {object}  dto.WebResponse{data=string} "URL PDF Tata
Tertib sukses disimpan"
// @Failure     400         {object}  dto.WebResponse "Input tidak valid atau
format bukan PDF"
// @Failure     404         {object}  dto.WebResponse "Konser tidak ditemukan"
// @Failure     500         {object}  dto.WebResponse "Kesalahan Server"
// @Router      /concerts/{id}/rules [post]
func (h *ConcertHandler) UploadRulesPDF(c *gin.Context) { ... }
```

7. Tulis Deklarasi General API: Buka berkas utama `main.go` dan tambahkan metadata umum:

```
// @title        Sistem Tiket Konser API
// @version      1.0
// @description  Dokumentasi API interaktif untuk sistem pemesanan tiket
konser.
// @termsOfService http://swagger.io/terms/

// @contact.name Developer JuaraCoding
// @contact.email support@juaracoding.com

// @securityDefinitions.apikey BearerAuth
// @in            header
// @name          Authorization
// @description   Masukkan token JWT dengan format: Bearer
<JWT_TOKEN>

// @host         localhost:8080
// @basePath     /api/v1
func main() { ... }
```

8. Integrasikan Rute Swagger UI: [routes/routes.go](#)

Buka berkas `routes/routes.go`, import berkas doc hasil generate swaggo, daftarkan rute UI, penyedia file statis, serta daftarkan rute upload thumbnail dan PDF tata tertib:

```
package routes

import (
    // Ganti 'go-tiket-konser' dengan nama module di go.mod Anda
    _ "go-tiket-konser/docs"

    "go-tiket-konser/handler"
    "go-tiket-konser/middleware"

    swaggerFiles "github.com/swaggo/files"
    ginSwagger   "github.com/swaggo/gin-swagger"
    "github.com/gin-gonic/gin"
    "gorm.io/gorm"
)

func SetupRouter(db *gorm.DB) *gin.Engine {
    r := gin.Default()

    // 1. Ekspos direktori 'uploads' secara statis ke rute URL /uploads
    r.Static("/uploads", "./uploads")

    // 2. Daftarkan Endpoint Swagger Web UI
    r.GET("/swagger/*any", ginSwagger.WrapHandler(swaggerFiles.Handler))

    // 3. Daftarkan Rute Endpoint Upload Thumbnail & PDF Tata Tertib
    api := r.Group("/api/v1")
    {
        api.POST("/concerts/:id/thumbnail", concertHandler.UploadThumbnail)
        api.POST("/concerts/:id/rules", concertHandler.UploadRulesPDF)
    }

    // ... pendaftaran routes api/v1 lain tetap dipertahankan ...

    return r
}
```

Jalankan perintah ini di root folder proyek untuk menghasilkan folder `docs/`:

```
swag init
```

Setelah itu, jalankan aplikasi `go run main.go` dan buka tautan berikut di browser Anda: `http://localhost:8080/swagger/index.html`.

Sesi 5: Tantangan Praktikum Mandiri (Tugas Mandiri)

Selesaikan tantangan berikut untuk memantapkan keahlian Anda dalam merancang sistem pengiriman email teratur dan standardisasi dokumentasi API.

Tantangan 1: Mengubah Pengiriman Email Menjadi Antrean Worker (Goroutine + Channel)

Pemicuan goroutine secara langsung menggunakan perintah `go func()` memiliki risiko kehilangan data (misalnya jika server mengalami crash mendadak di tengah jalan, email akan hilang selamanya dari memori).

- **Tugas:**

1. Buat struktur penampung pekerjaan baru bernama `EmailJob` di dalam file `service/email.go` yang memuat field: `To`, `CustomerName`, `BookingCode`, dan `TotalAmount`.
2. Inisialisasi sebuah channel penampung pekerjaan global bertipe buffered channel: `EmailQueue chan EmailJob` dengan kapasitas buffer 100.
3. Tulis fungsi inisialisasi worker pool `StartEmailWorkers(workerCount int)` yang meluncurkan sejumlah `workerCount` goroutine independen yang terus menerus mendengarkan input dari channel `EmailQueue`:

```
func StartEmailWorkers(workerCount int, emailService EmailService) {
    for i := 0; i < workerCount; i++ {
        go func() {
            for job := range EmailQueue {
                _ = emailService.SendTicketConfirmation(job.To,
                    job.CustomerName, job.BookingCode, job.TotalAmount)
            }
        }()
    }
}
```

4. Ubah pemanggilan pengiriman email di `service/booking.go`. Alih-alih meluncurkan goroutine langsung, kirimkan data pekerjaan ke dalam channel:

```
EmailQueue ← EmailJob{To: customer.Email, ...}
```

Tantangan 2: Dokumentasi Swagger Lengkap untuk Modul Autentikasi dan Registrasi

Lengkapi berkas dokumentasi Swagger interaktif Anda agar mencakup modul registrasi, login, dan logout dengan kriteria keamanan JWT.

- **Tugas:**

1. Tuliskan anotasi Swaggo lengkap di atas fungsi `Register` dan `Login` di dalam `handler/auth.go`.
2. Tambahkan anotasi `@Security BearerAuth` di atas fungsi-fungsi terproteksi (seperti `Logout`, `GetProfile`, dan `CreateBooking`) agar antarmuka web UI Swagger memunculkan tombol ikon gembok ("Authorize") untuk memasukkan token JWT secara interaktif saat diuji.
3. Lakukan generate ulang spesifikasi melalui `swag init`, lalu verifikasi hasilnya pada browser Anda.